EXPERIMENT: 05

Write a C program for generalization of the Caesar cipher, known as the affine Caesar cipher, has the following form: For each plaintext letter p, substitute the ciphertext letter C: C = E([a, b], p) = (ap + b) mod 26 A basic requirement of any encryption algorithm is that it be one-to-one. That is, if p q, then E(k, p) E(k, q). Otherwise, decryption is impossible, because more than one plaintext character maps into the same ciphertext character. The affine Caesar cipher is not one-to-one for all values of a. For example, for a = 2 and b = 3, then E([a, b], 0) = E([a, b], 13) = 3. a. Are there any limitations on the value of b? b. Determine which values of a are not allowed.

CODE:

```
#include <stdio.h>
#include <ctype.h>
int gcd(int a, int b)
{
  while(b != 0)
  {
    int temp = b;
    b = a % b;
    a = temp;
  }
  return a;
}
int main()
{
  char text[100];
  int a, b, i;
  printf("Enter the text: ");
  fgets(text, sizeof(text), stdin);
  printf("Enter value of a: ");
  scanf("%d", &a);
  printf("Enter value of b: ");
  scanf("%d", &b);
  if(gcd(a, 26) != 1)
  {
```

```
    printf("Invalid value of a!\n");
    printf("a must be relatively prime to 26.\n");
    return 1;
  }
  b = b % 26;
  for(i = 0; text[i] != '\0'; i++)
  {
    if(isupper(text[i]))
    {
      text[i] = ((a * (text[i] - 'A') + b) % 26) + 'A';
    }
    else if(islower(text[i]))
    {
      text[i] = ((a * (text[i] - 'a') + b) % 26) + 'a';
    }
  }
  printf("Encrypted text: %s", text);
  return 0;
}
```

OUTPUT:

